



Zhihao Li's
Pocket Book

From 2024
to 2025

Professional Intuitions

Zhihao Li

Computer Science and Technology, Xidian University

zhihaoli@stu.xidian.edu.cn

CONTENTS

CHAPTER 1	PROFESSION & LEARNING	PAGE 4
1.1	Mathematical Principles	4
1.2	Computer Graphics	4
1.3	Digital Human	4
CHAPTER 2	PHILOSOPHY & INSIGHT	PAGE 5
2.1	Research Guidelines	5
2.1.1	Global Stages	5
2.1.2	Early Stage	5
CHAPTER 3	VIEWS & INSPIRATION	PAGE 6
3.1	Learning With Noisy Labels	6
CHAPTER 4	CODING & SKILLS	PAGE 7
4.1	Environment Configuration	7
4.1.1	<i>Miniconda</i> Installation On Linux	7
4.1.2	<i>Pytorch3d</i> Installation on Windows	7
4.1.3	<i>Ninja</i> Installation on Linux	8
4.2	Python Debugging	8
4.2.1	<i>Visual Studio Code</i> Debugging Configuration	8
4.3	Bugs with Solution	9
4.3.1	Version Conflict	9
4.4	Coding Tricks	10
4.4.1	Python	10
4.4.2	Cmd On Windows	10
CHAPTER 5	GRAMMAR & WRITINGS	PAGE 11
5.1	Mathematical Symbols in LaTeX	11
5.1.1	集合及向量空间的表示	11
5.1.2	运算符号	11

5.2	Extended Markdown	11
5.3	Elaborating Ideas	12

Chapter 1

Profession & Learning

1.1 Mathematical Principles

1. 仿射变换 (Affine Transformation)

$$price = w_{area} \cdot area + w_{age} \cdot age + b \quad (1.1)$$

仿射变换的特点是通过加权和对特征进行线性变换，并通过偏置项进行平移。

2. 非线性频率压缩

在滤波器设计中将整个模拟频率轴压缩到 π/T 之间，使得 $H_a(s), s = j\Omega$ 压缩为 $\widehat{H}_a(s_1), s_1 = j\Omega_1$ ，可以利用正切变换实现频率压缩模型：

$$\Omega = \frac{2}{T} \tan\left(\frac{1}{2}\Omega_1 T\right) \quad (1.2)$$

这个设计思想实质上利用了正切函数定义域有限、值域无限以及奇函数的性质；推而广之，这种设计可以实现特定的单值压缩方法，也可以实现值域的延展。

一些类似的函数特性，对数函数，指数函数分别适合于定义域、值域取值 $0 \sim 1$ 之间的情况，但是对目标域都有所限制，因此这些函数往往没有正切函数具有优良的特性。

1.2 Computer Graphics

1.3 Digital Human

Chapter 2

Philosophy & Insight

2.1 Research Guidelines

2.1.1 Global Stages

1. 不要被工程化的思维所束缚，做 A+B 的工作，着重分析本质，从源头创新。

2.1.2 Early Stage

1. 前期注重多看研究论文，精度并总结，打牢基础，做研究综述。
2. [研究综述](#)：有多少途径能够实现研究目的，各自是从哪些地方出发的，本质的区别是什么。

Chapter 3

Views & Inspiration

3.1 Learning With Noisy Labels

1. Co-teaching: Robust Training of Deep Neural Networks with Extremely Noisy Labels

- Views: DNN 的相互指导学习机制，两个模型分别动态地选取一些干净样本相互提供给对方进行学习，目的是过滤不同类型的噪声；
- Inspiration: 直觉是同辈相互交错的学习机制，而且训练中其样本选取的动态性值得一提；

2. DivideMix: Learning with Noisy Labels as Semi-supervised Learning

- Views: 对噪声数据集进行划分，并同时学习两个模型进行相互指导、标签集成，以克服不同类型的噪声；
- Inspiration: Mutual Learning 和交互学习一定程度上可以增强模型鲁棒性；

Chapter 4

Coding & Skills

4.1 Environment Configuration

4.1.1 Miniconda Installation On Linux

```
1  # Fetch the miniconda script
2  export HOME=$PWD
3  wget -q https://repo.anaconda.com/miniconda/ \
4  Miniconda3-py37_4.12.0-Linux-x86_64.sh -O miniconda.sh
5  sh miniconda.sh -b -p $HOME/miniconda3
6  rm miniconda.sh
7  export PATH=$HOME/miniconda3/bin:$PATH
8
9  # Initialize conda
10 source $HOME/miniconda3/etc/profile.d/conda.sh
11 hash -r
12 conda config --set always_yes yes --set changeps1 yes
13
14 # Create new environment
15 conda create -n my_env python=3.8
16 conda activate my_env
```

4.1.2 Pytorch3d Installation on Windows

Windows 下 3D Vision 环境配置 (GaussianAvatars)

4.1.3 Ninja Installation on Linux

```
1 apt install ninja-build
```

4.2 Python Debugging

4.2.1 Visual Studio Code Debugging Configuration

为了能够在 Visual Studio Code 中调试 Python 项目，首先需要安装 Python 和 Python Debugger 扩展。然后如图4.1所示，点击左侧的调试按钮，添加 launch.json 文件，并根据代码的配置示例创建项目配置。

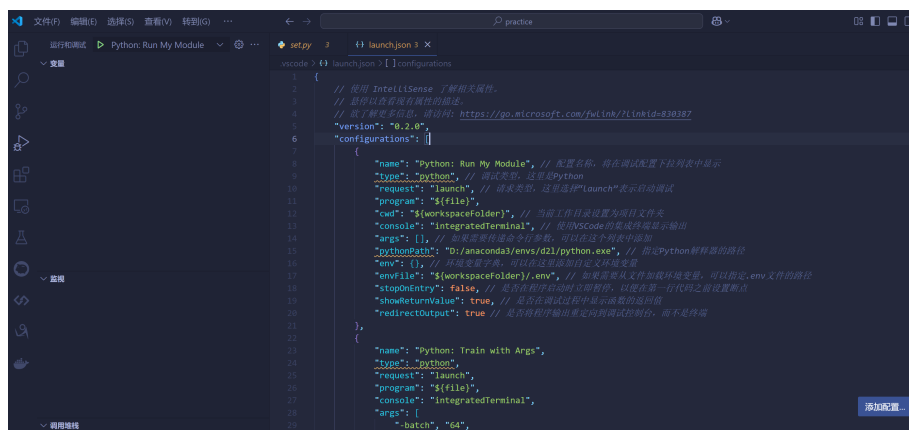


Figure 4.1: Python 项目调试配置

```
1 {
2     "version": "0.2.0",
3     "configurations": [
4         {
5             // 配置名称，将在调试配置下拉列表中显示
6             "name": "Python: Run My Module",
7             "type": "python", // 调试类型，这里是 Python
8             "request": "launch", // 请求类型，这里选择 "launch" 表示启动调试
9             "program": "${file}",
10            "cwd": "${workspaceFolder}", // 当前工作目录设置为项目文件夹
11            // 使用 VSCode 的集成终端显示输出
12            "console": "integratedTerminal",
13            "args": [], // 如果需要传递命令行参数，可以在这个列表中添加
14            // 指定 Python 解释器的路径
15            "pythonPath": "D:/anaconda3/envs/d2l/python.exe",
16            "env": {}, // 环境变量字典，可以在这里添加自定义环境变量
17            // 如果需从文件加载环境变量，可以指定 .env 文件的路径
```



```

18     "envFile": "${workspaceFolder}/.env",
19     // 是否在程序启动时立即暂停，以便在第一行代码之前设置断点
20     "stopOnEntry": false,
21     "showReturnValue": true, // 是否在调试过程中显示函数的返回值
22     "redirectOutput": true // 是否将程序输出重定向到调试控制台，而不是终端
23 }
24 ]
25 }

```

在完成以上准备工作后，设置程序端点，然后直接点击调试三角形按钮，即可开始调试，如图 4.2 所示。

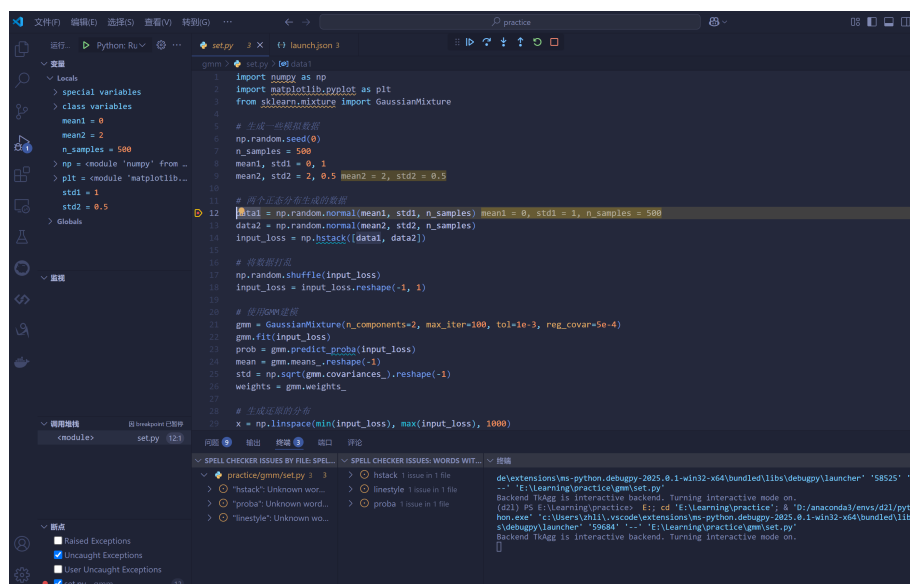


Figure 4.2: Python 程序调试示例

4.3 Bugs with Solution

4.3.1 Version Conflict

- Q: Numpy 版本和 Pytorch 版本不兼容。
 - A: 尝试更换 Numpy 版本。

```

1     assert (self.faces==torch.from_numpy(flame_model.f.astype( \
2         'int64'))).all()
3     RuntimeError: Numpy is not available

```

- Q: PIL 版本更新问题。
 - A: 在 pillow 的 10.0.0 版本中,ANTIALIAS 方法被删除了,通过命令 `pip install Pillow==9.5.0` 安装旧版本即可。

```
1 AttributeError: module 'PIL.Image' has no attribute 'ANTIALIAS'
```

4.4 Coding Tricks

4.4.1 Python

1. 路径管理库 pathlib

- 类型指定: `input: Path`
- 路径赋值: `'../../dir'`
- 判断路径是否存在: `Path.exists()`
- 获取父目录: `Path.parent`
- 通配符匹配: `Path.glob()`

2. 数字可读性 8_000: Python 中数字表示含有下划线时, 仅用于提高大数字的可读性, 对数值没有影响。

4.4.2 Cmd On Windows

1. 打印当前目录路径 pwd

```
1 PS D:\ZHaoLi\02.Hobbies\01.Blog> pwd
2
3 Path
4 ----
5 D:\ZHaoLi\02.Hobbies\01.Blog
```

Chapter 5

Grammar & Writings

5.1 Mathematical Symbols in LaTeX

5.1.1 集合及向量空间的表示

Rendered Symbol	Source Code	Rendered Symbol	Source Code
S	<code>$\mathcal{S}$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.1: LaTeX Symbols and Their Source Code

5.1.2 运算符号

Rendered Symbol	Source Code	Rendered Symbol	Source Code
$\ll$	<code>$\ll$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.2: LaTeX Symbols and Their Source Code

5.2 Extended Markdown

在 Markdown 文件中实现下拉式列表，可以通过 ‘html’ 语言实现，具体语法如下：

```
1 <details>
2   <summary> Dependencies (click to expand) </summary>
3
4   ## Dependencies
5   - PyTorch 1.4
6   - matplotlib
7   - numpy
8   - imageio
```

- imageio-ffmpeg
- configargparse

The LLFF data loader requires ImageMagick.
</details>

5.3 Elaborating Ideas

1. 首先阐述问题的必要性，然后引出自己的方法。

Chapter 6

Templates & Tools